

SEUPD@CLEF: Team RACOON

Giovanni Gaio¹, Filippo Mazzarotto¹, Matteo Meneghin¹, Enrico Saro¹ and
Francesco Visonà¹

¹University of Padua, Italy

Abstract

In recent years, the *Conference and Labs of the Evaluation Forum (CLEF)* has played a key role in advancing research on the temporal robustness of *Information Retrieval (IR)* systems. This paper presents the work of the Racocon group from the University of Padua on the LongEval CLEF 2025 Lab in Task 1 "LongEval-Web Retrieval" [1]. The task was focused on developing IR systems resilient to the temporal evolution of Web documents. To address this challenge, our approach combines traditional IR techniques with several novel components. Specifically, our *Search Engine (SE)* incorporates a French lemmatizer within the analyzer, a *Large Language Model (LLM)*-based *Query Expansion (QE)* module, a *Relevance Feedback (RF)* mechanism based on query assessment over the preceding months, and an LLM-based Elo reranking strategy. Overall, our approach based on RF combined with *Deep Learning (DL)* reranking demonstrates substantial improvements over the baseline.

Keywords

Information Retrieval, Search Engine, Temporal Evolution, Relevance assessment, Query expansion, Reranking, Large Language Model, Conference and Labs of the Evaluation Forum

1. Introduction

A SE is a software system designed to retrieve information from a collection of documents, based on user search queries. While this task has been addressed largely in literature through various approaches, maintaining robust performance amid dynamic temporal changes in textual content remains an open challenge.

In this work, we set out to develop an IR system capable of sustaining its performance over time. This was conducted within the framework of the first task "LongEval-Web Retrieval" of the "LongEval" Laboratory proposed by CLEF 2025¹. The dataset consisted of a sequence of web document collections and queries, primarily in French, provided by Qwant², that spanned over nine months, capturing the evolution of documents and queries over time.

Our objective was to explore a number of different approaches, mostly standard techniques in various combinations, but also original ideas of our own. We started with a baseline SE setup based on Lucene [2], and incrementally extended it by experimenting with different configurations for analyzing the documents and, more significantly, with a range of strategies aimed at improving their retrieval. As LLMs have gained widespread attention, praise, and adoption in recent times, we were particularly interested in evaluating their potential integration within our pipeline. We were ultimately limited by our computational resources in the depth of our tests, but we were nonetheless able to obtain several interesting results.

The paper is organized as follows: section 2 discusses the ideas we were inspired by; section 3 describes our approach; section 4 explains our experimental setup; section 5 discusses our main findings and compares the various approaches; finally, section 6 draws some conclusions and outlooks for future work.

¹"Search Engines", course at the master degree in "Computer Engineering", Department of Information Engineering, and at the master degree in "Data Science", Department of Mathematics "Tullio Levi-Civita", University of Padua, Italy. Academic Year 2024/2025

✉ giovanni.gao.1@studenti.unipd.it (G. Gaio); filippo.mazzarotto@studenti.unipd.it (F. Mazzarotto);
matteo.meneghin@studenti.unipd.it (M. Meneghin); enrico.saro@studenti.unipd.it (E. Saro);
francesco.visona.3@studenti.unipd.it (F. Visonà)



© 2025 Copyright for this paper by its authors.
Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

¹For further information see: <https://clef-longeval.github.io/tasks/>.

²<https://about.qwant.com/en/>

2. Related Work

The foundation of our work builds upon the code provided by Professor Ferro and tutor Andrea Pasin in the Search Engines course held at the University of Padua. This preliminary implementation was subsequently refined, incorporating insights from various approaches presented in prior studies that addressed the problem of longitudinal evaluation of IR systems, particularly within the CLEF 2024 LongEval Laboratory.

Specifically, the ideas of *Leveraging Prior Relevance Signals in Web Search* [3] were of particular interest to us, as they focused on the use of relevance assessments as a form of QE or reranking to obtain better performances over time. Their work described how these methods could improve the mean nDCG of up to 8%.

We also focused on the methodology of Team Mouse in 2024 [4] which described how to use LLMs for QE and reranking through a reranker API. The latter approach was developed but not used due to constraints on the rate limits of such services.

The prompt design for the QE phase, discussed in subsection 3.3.1, was primarily driven by a trial-and-error approach, drawing inspiration from the works *Query Expansion by prompting Large Language Models* [5] and *Corpus-Steered Query Expansion with Large Language Models* [6].

3. Methodology

This section describes the architecture of our system, which was developed primarily in Java using the *Apache Lucene* library, and in Python for some specific tasks of the Searcher component.

Our initial objective was to implement a fully functional pipeline, which was subsequently extended to improve performance. The system is structured into four main components:

- **Parser:** a standard JSON parser designed for the document collection. It uses the *Jackson* library³.
- **Analyzer:** responsible for text processing of the documents, including tokenization and the application of filters. It is described in subsection 3.1.
- **Indexer:** constructs an index of the tokens obtained from the parsed and analyzed documents. It contains the document IDs and the terms of their bodies, other than a full text field that is used in the reranking phase, as described in subsection 3.2.
- **Searcher:** manages query processing and document retrieval by adopting some strategies like QE, RF and reranking. See subsection 3.3 for further details.

The following sections provide a more detailed description of the system, with a focus on the components that have been most substantially improved. A diagram of the overall configurable architecture is shown later in Figure 2.

3.1. Analyzer

The analyzer is the component which is responsible for analyzing the raw text of the documents and turning it into a set of tokens that can be used for indexing and searching.

During our tests we tried different combinations of filters and tokenizers. The final pipeline is shown in Figure 1 and described below.

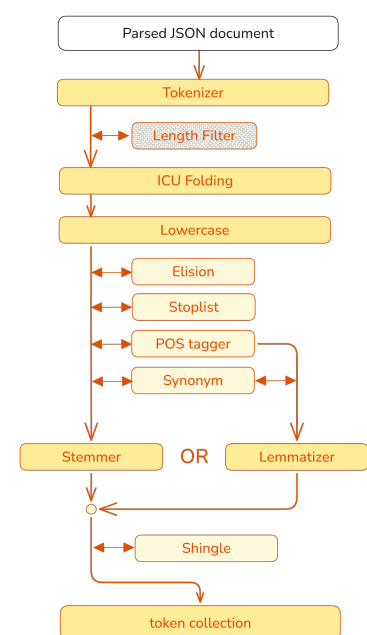


Figure 1: RacoonAnalyzer Pipeline.

³<https://github.com/FasterXML/jackson>

3.1.1. Tokenizer

The first component of our pipeline is the tokenizer, which splits the text of each document into smaller units, producing a stream of tokens.

We tested three built-in tokenizers in *Apache Lucene*:

- **StandardTokenizer**: implements the Word Break rules from the Unicode Text Segmentation algorithm, as specified in Unicode Standard Annex 29 [7].
- **WhitespaceTokenizer**: divides text at any whitespace character.
- **LetterTokenizer**: divides text at non-letter characters.

3.1.2. Filters

The remaining pipeline was composed of various filters and the stemming step. The latter is described in more detail in subsection 3.1.3. The rest of the filters are now described in the order they were applied:

1. *Length token filter (optional)*: filters out tokens that are too short or too long, given a minimum and maximum length.
2. *ICU folding*: applies search term folding to Unicode text, such as accent and diacritic removal.
3. *Lowercase*: converts all the text to lower case, which is better for information retrieval.
4. *Elision filter (optional)*: filters out elided tokens, such as *d'*.
5. *Stopword filter (optional)*: filters out stop words, such as articles, prepositions and conjunctions, based on an external list of words.
6. *POS tagging (optional)*: tags each token with its part of speech.
7. *Synonyms expansion (optional)*: expands each token with synonym, which are obtained from the WOLF database [8]. During its test, we moved the *ICU Folding* filter after this step.
8. *Shingle filter (optional)*: a Word N-gram filter that creates overlapping sequences of tokens and treats them as an index term.

3.1.3. Stemming and its alternatives

A key step of the pipeline is the normalization through stemming or lemmatization. Stemming is the process of reducing words to their root form, while Lemmatization is the process of reducing words to their base dictionary form. We tested several built-in alternatives *Apache Lucene* provides, as well as a custom lemmatizer based on a large-scale French lexicon. The built-in solutions are:

1. **Light**: Applies a light stemming algorithm specifically designed for the French language, performing less aggressive suffix stripping than more intensive stemmers.
2. **Minimal**: Implements a minimal stemming algorithm for French, which typically removes only the most common inflections.
3. **Snowball**: Utilizes the Snowball stemming algorithm (often an evolution of the Porter stemmer) with rules tailored for the French language.
4. **NGram**: Applies a character-level N-gram token filter. This method breaks tokens into sequences of 'n' characters. It is distinct from stemming as it does not aim to find the morphological root form of a word but instead represents tokens through their constituent character n-grams.

Our **Lexicon lemmatizer** is based on the *Lexique des formes fléchies du français (LEFFF)* [9], a large-scale morphological and syntactic lexicon for French. It contains roughly 600,000 entries, which map inflected forms to their lemmas, covering inflections for words (including multi-word entities) and punctuation.

Each LEFFF entry typically has four fields: the *inflection* (the word form as it appears in text), the *part of speech* (e.g., verb, noun), the *lemma* (the dictionary base form), and the inflected form's *grammatical*

features. For example, an entry like `'persistait,v,persister,I3s'` maps the word "persistait" to its lemma "persister" when "persistait" is identified as a verb (v).

Our lemmatizer uses each lexicon entry as a rule: for a given token and its pre-assigned *Part-of-Speech* (POS) tag, if a matching inflected form is found in the LEFFF lexicon, the token is replaced with the corresponding lemma. During our testing, we moved the *ICU Folding* filter (for accent and diacritic removal) after the lemmatization step, obtaining great results on example sentences.

However, the lemmatizer did not perform as well in practice. This is likely because real-world queries and document text are often noisy or contain variations not perfectly matching the strict entries in the LEFFF lexicon.

3.2. Indexing

The indexer is a standard way to index documents in memory. We used the `IndexWriter` of *Apache Lucene*, storing the IDs (not filtered) and the tokens of each document. We also stored the term vectors and the positions of each token to be able to retrieve the documents from the index, which is really useful for some components of the searcher, like RF. Since most of the rerankers we tested required access to the full document text, we allocated a non-indexed field within our index to store a compressed version of each document's full text. While this decision doubled the overall index size, it significantly improved performance during the reranking phase.

We also considered the idea to store the corresponding URLs of the documents, since many queries are incomplete URLs or refer to them, but decided against it given the poor results they had on the collection of the previous year (see for example [10]).

Given that this process is highly parallelizable, we used multiple threads to index the documents in parallel, so that the index step takes only about 2 minutes to finish for each month of the training set. In this phase it was critical to notice that many of the documents were duplicates, and subsequently store only one copy of them.

3.3. Searcher

The searcher is the component that processes queries and retrieves documents from the index, which means it has a pivotal role in IR systems. That is why we sought to make it as fast as possible: we implemented a custom searcher that uses the `IndexSearcher` of *Apache Lucene* to retrieve the relevant documents, executing one thread for each query.

In the standard searching algorithm we tried to achieve a good balance between exact matches, highly similar results, and proximity-based queries, using the BM25 Similarity [11]. Other than the standard searching, which was implemented as a *Boolean Query*, we followed different ways to achieve better performances:

1. QE, described in subsection 3.3.1.
2. RF, described in subsection 3.3.2.
3. Reranking, described in subsection 3.3.3.

3.3.1. Query expansion

To enrich user queries with semantically related terms and improve information retrieval performance, we implemented a QE pipeline using LLMs from Meta's LLaMA family, specifically LLaMA 3 70B [12] and LLaMA 4 Scout 17B [13], accessed through the Groq API⁴. The process aims to generate relevant query variations or semantic enrichments to boost recall in downstream retrieval components.

⁴<https://console.groq.com>

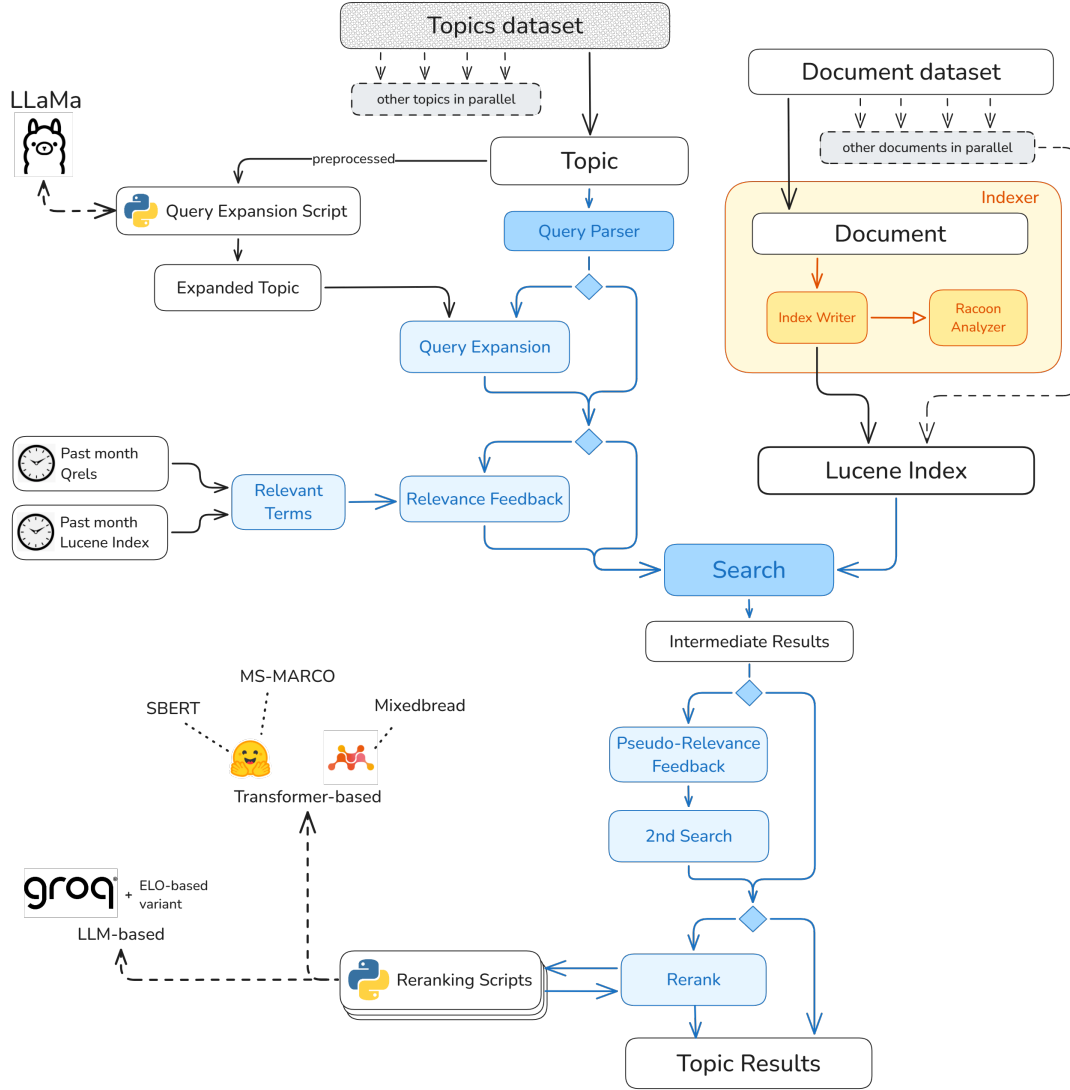


Figure 2: The architecture of our system, indexing documents in parallel using the `RacoonAnalyzer` and then searching all the topics. Diamonds represents conditional configurations.

Prompt Design. The prompt design process was primarily driven by a trial-and-error approach, drawing inspiration from the works *Query Expansion by prompting Large Language Models* [5] and *Corpus-Steered Query Expansion with Large Language Models* [6].

Prompts were formulated in French to align with the language of the query logs. Ultimately, four distinct prompts were developed to explore different prompting strategies and to identify the best approach:

1. Generating 20 semantically related or contextually adjacent expressions to a query.
“Étendre la requête suivante avec 20 termes ou expressions liés en French: {query}. Imprimez uniquement les termes sur une seule ligne, séparés par des virgules, n’ajoutez pas de texte ou d’explications supplémentaires.”
2. Providing a passage that answers the query, along with a justification.
“Rédigez un passage qui réponde à la question posée {query} Justifiez votre réponse avant de répondre. N’ajoutez pas de texte supplémentaire”
3. Listing synonyms and conceptually related terms.

“Vous êtes un assistant de recherche d’informations. Etant donné la requête {query}, générez une liste de 10 termes sémantiquement liés, de synonymes et de mots-clés conceptuellement adjacents qui pourraient être utilisés pour étendre cette requête afin d’obtenir une meilleure couverture. Imprimez uniquement les termes sur une seule ligne, séparés par des virgules, n’ajoutez pas de texte ou d’explications supplémentaires.”

4. Inferring the user’s intent explicitly.

*“Vous trouverez ci-dessous une requête écrite par un utilisateur français à un moteur de recherche. Expliquez quelle est l’intention de la personne qui l’a écrite et répondez à la requête s’il s’agit d’une question. Commencez par ‘L’utilisateur veut’. N’ajoutez pas de texte supplémentaire
Requête: {query}”*

All prompts instructed the LLM to avoid adding any additional text, in order to minimize noise in the output. In Prompt 4, the model was explicitly asked to begin its response with “L’utilisateur veut”, to prevent variations that might introduce undesired leading terms. These initial words were subsequently filtered out.

Models and APIs. We used the llama3-70b-8192 model as the primary generator due to our constraints on rate limits, and meta-llama/llama-4-scout-17b-16e-instruct as a fallback for ambiguous or error-prone completions. Only when all these models failed, which means with really explicit queries, our system didn’t find any expansion term. Responses were streamed from the Groq API with temperature 1.0 and a maximum of 1024 tokens, which allowed us a restricted number of queries per day and per minute: that’s the main reason for the implementation the caching strategy.

Caching and Efficiency. Since the expansion process is time-consuming and subject to rate limit constraints, we implemented a caching strategy to prevent latency at search time: we asked the LLM to expand the queries and we stored the results in output files indexed by month and prompt type before running our system. Although this may seem unrealistic in real-life applications, it wouldn’t be needed if the limits on the Groq API weren’t so strict. These limits were respected using delays and token usage monitoring via response headers.

Weights of expanded terms. In this section, but also in the RF one, we tried different ways to weight our expanded terms.

- **Constant weight:** the same weight is given to all the terms, which after some experiments was set to 0.5.
- **Position weight:** the weight of term i , where i refers to the position of the term in the expanded sequence, is calculated as:

$$w_i = \max \left(0.1, 0.5 \cdot \left(1 - \frac{i}{E} \right) \right)$$

where E is the total number of expanded terms. This formula was primarily focused on the prompts asking for related terms, with the implicit assumption that the first ones were the most relevant.

- **IDF-like weight:** the weight of term i is calculated as:

$$w_i = 0.1 + \min \left(0.4, \frac{\log (N/(n_i + 1))}{10} \right)$$

where n_i is the number of documents containing term i , N is the total number of documents, and E is the total number of expanded terms. The formula was obtained by modifying the smoothed inverse document frequency dividing it by 10, while also ensuring a minimum boosting of 0.1 and a maximum of 0.5.

3.3.2. Relevance feedback

The main focus of the task was to understand how our system could leverage the knowledge of previous months relevance to handle changes over time. Although the use of relevance assessments may seem unrealistic in practice, in a production environment they could be easily derived from implicit feedbacks like clicks, as it was done to build the CLEF LongEval training collection, or from explicit assessments of real users.

Our work, which was based on the ideas of *Leveraging Prior Relevance Signals in Web Search* [3], aims to use the information of relevance of prior months for a given query to boost relevant results for the same query in the current month. While this method is limited to topics with a known history, in this specific task the great overlapping between queries of different months allows the method to be very effective.

Related work. The focus of [3] is on two different approaches: on the one hand, it uses relevance assessments as a form of QE, adding terms of prior relevant documents to the query itself. The other approach focuses on the use of relevance assessments in the reranking phase to boost relevant results in the current month by weighting the presence of documents in prior evaluations based on the difference in time (older relevant documents gets a lower weight than newer ones).

Our work. We developed both of the approaches of [3]: the first one is described below, while the reranking method can be found in subsection 3.3.3.

The following QE methods were tested:

- *Pseudo RF (PRF)*: This is a standard PRF approach. We perform the standard query, and then we retrieve the 10 most frequent terms in the 5 most relevant documents and add them to the query itself.
- *RF*: This is a more complex approach, where before performing any retrieval we try to see if the query itself has already been made in the previous months. If so, we get the 8 most relevant documents for that query and add their 20 top terms to the query itself, confident that what was relevant in the previous months can guide the same query in the current month.
- *RF and PRF*: This combines the two previous approaches to integrate the strengths of both methods.

All the above numbers were obtained after a fine tuning phase on the training dataset.

As described in subsection 3.3.1, we tried different weights to give to the new terms added to the query:

- **Constant weight**
- **Position weight**
- **IDF-like weight**

The best results for RF were obtained using a constant weight of 0.5.

3.3.3. Reranking

After retrieving relevant documents, which were already given a score using the Okapi BM25 ranking function, we took the highest-scoring ones and updated their relevance using a more complex and expensive procedure. This is a standard step in modern IR systems, so we had a lot of works to take ideas from. The number of reranked documents is in some cases ultimately limited by the hardware available to us.

SBERT vector embedding cosine similarity score We modified the previously computed score for each retrieved document by adding the cosine similarity between the vector embeddings of the document and the query. These embeddings were calculated using a pretrained model available in the python library `sentence_transformers` [14, 15].

We tried different models from this library, but, given the limited time and computational power at our disposal, we selected `sentence-transformers/distiluse-base-multilingual-cased-v1` as a good "standard" model, as it is big and multilingual while also not being too slow, thus allowing us to re-rank 100 documents per query by simply adding the new similarity scores multiplied by 1.5.

Text Ranking models We also did some tests using two more models from the same library: `cross-encoder/ms-marco-MiniLM-L6-v2`⁵ and `mixedbread-ai/mxbai-rerank-base-v1`⁶ [16].

These models are specifically aimed at reranking and directly provide a similarity score between the topic and document provided. While the first one is smaller and faster, allowing to re-rank 100 documents, the second one is bigger and slower, so we set it to re-rank only 60 documents. Also in this case we multiplied the similarity scores by 1.5.

LLM-based Elo pseudo ranking This approach is based on the assumption that LLMs, like humans, are better at comparing things than scoring. The idea is to sort the documents by comparison (it is a sort of pairwise approach, in contrast to the previous pointwise approaches). The main difficulty is that doing a complete sorting requires a number of comparisons in the order of $\mathcal{O}(n \log n)$, which is too many (at least given our limited resources).

For these reasons we tried to update the scores using an Elo ranking scheme. We selected some random documents and asked an LLM to rank them by relevance to the given topic.

We then update the score S for each document according to the following rule:

$$S_i^{\text{new}} = S_i^{\text{old}} + k(R - E)$$

Where E is the expected number of comparisons where the document i ranked better than the other; R is the actual number of comparisons where the document was better, for the ordering given by the LLM.

For each comparison we estimate the probability that a document d_A is better than document d_B , with respective scores S_A and S_B , is $\mathbb{P}(d_A > d_B) = \frac{1}{1 + \exp(0.5 \cdot (S_A - S_B))}$. Then to find R we just sum over all documents: $E_A = \sum_{d \in D \setminus \{d_A\}} \mathbb{P}(d_A > d)$.

We were planning on using some freely available API to answer the prompt. This was ultimately not possible because the method requires a large number of queries and long prompts. We also tried using smaller models on the hardware we had. Also this approach was not feasible, thus we could not evaluate it.

Relevance feedback reranking. This approach is based on what was explained in subsection 3.3.2. The idea is to use the query assessments from previous months to re-rank the retrieved documents: if a document was relevant in the past, we assign it a higher score. However, this strategy can easily lead to stale results, therefore we introduce a weight that favors more recent assessments, following the formula:

$$S_i^{\text{new}} = S_i^{\text{old}} + \alpha \cdot 0.8^{M_{diff}}$$

where M_{diff} refers to the difference between the current month and the month of the last relevance assessment, while α is a value that depends on the prior relevance of the document: 1.0 if the relevance was equal to 2, 0.5 if the relevance score was 1.

⁵<https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2>

⁶<https://huggingface.co/mixedbread-ai/mxbai-rerank-base-v1>

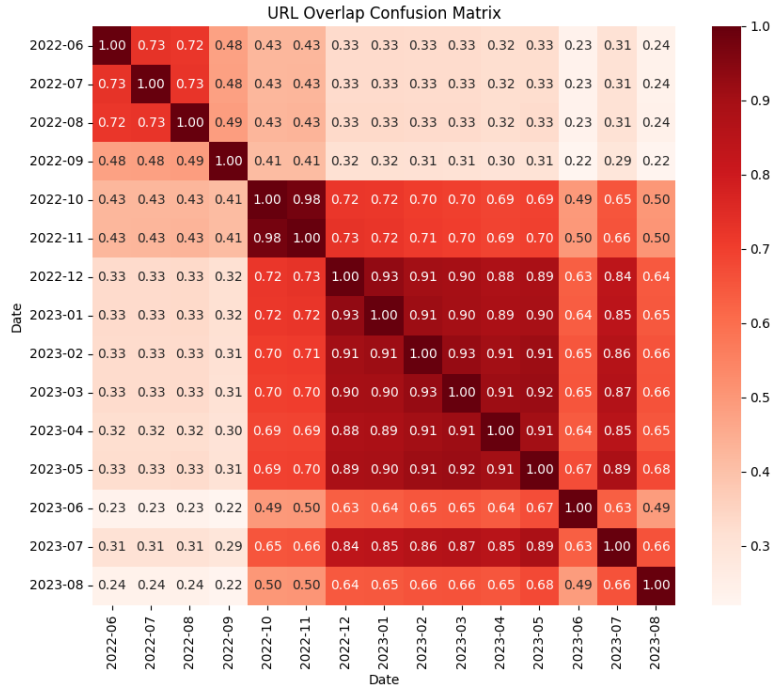


Figure 3: Overlap matrix of the documents in the train collection. Available at <https://clef-longeval.github.io/data/>

API reranking. This approach is based on different services that allow to re-rank automatically documents for a given query by simply passing the query and the top documents. While this method could be really useful, we couldn't use it due to the high volume of queries to process and the limited rate limits of the services. Nevertheless, it could be really interesting to try in the future.

4. Experimental Setup

The source code of our information retrieval system is available online at the following address: <https://bitbucket.org/upd-dei-stud-prj/seupd2425-racoon/src/master/>.

Hardware setup The SE and all suitable tests were run on the following system:

- *Operating System (OS):* Windows 11 Home,
- 32 GB of RAM,
- AMD Ryzen 9 6900HX with Radeon Graphics. The absence of a dedicated NVIDIA GPU is a limitation, which was overcome by running some parts of the reranking phase (see subsection 5.3) on a separate computer.

To allow easy and fast use of LLMs, we also took advantage of the online service groq.com, this was done mostly for QE. We mostly used Llama 3 70B as LLM, while occasionally asking for Llama 4 17B Scout to expand queries with an explicit content.

In some cases, to accelerate inference with machine learning models, we ran just this part on another computer. This second computer communicated via *HyperText Transfer Protocol (HTTP)* with the master computer described above and was equipped with an Nvidia GeForce GTX 960 with 4GB VRAM, 16 GB RAM, an Intel Core i5 6500 CPU, and running the OS Manjaro Linux.

Train collection The training data was provided by the organizing committee of CLEF 2025 [17], and is publicly available on the TU Wien Research Data Repository [18]. It consists of 9 monthly data snapshots, acquired from June 2022 to February 2023 composed of documents (18M), queries (9k) and qrels (relevance assessments). Both the documents and the queries vary in the 9 snapshots, but there is a high correlation between different months, both in the documents, as shown in Figure 3, and in the queries. The queries were extracted from Qwant’s search logs and based on a set of selected topics, while the documents are all the web pages that have been displayed in *Search Engine Result Pages (SERPs)* for the selected queries. Filters were applied to exclude adult content from the collection, but this also caused some problems in the relevance assessments, as described in section 5. The relevance estimates are obtained through automatic collection of user implicit feedback and are on a 3-level scale. For more information please refer to <https://clef-longeval.github.io/data/>.

Evaluation measures The submitted systems will be evaluated using the *Normalized Discounted Cumulated Gain (nDCG)* and *Relative nDCG Drop (RnD)* between different snapshots of the test set. The nDCG is a well-known evaluation measure calculated on a set of queries Q as follows [19]:

$$\text{nDCG}(r) = \frac{1}{|Q|} \sum_{j=1}^n Z_{kj} \sum_{m=1}^k \frac{2^{R(j,m)} - 1}{\log_2(1 + m)},$$

where $R(j, d)$ is the relevance score given to document d for query j ; Z_{kj} is a normalization factor, chosen so that a perfect ranking has nDCG equal to 1; k is a fixed positive integer indicating the number of documents evaluated in the ranking, if the ranking contains less than k documents, the sum is truncated.

The RnD is measured by computing the difference of nDCG between snapshots. This measure supports the evaluation of the impact of the data changes on the systems’ results.

Given that our systems will be evaluated on nDCG, all the experiments will focus on the nDCG measure, and make decision based on it. This doesn’t mean we didn’t look at other metrics, but we didn’t use them to make decisions.

Statistical analysis We conducted statistical analysis only on the queries that had nDCG different from 0.0 for at least one system. We first conducted a 2-way anova on the systems to determine if some of them were statistically different from the others, and then a Tukey HSD test to determine which systems were different. The confidence level was set to 0.05.

We report graphs of the results of Tukey HSD tests, while we didn’t report ANOVA results for the sake of space.

5. Results and Discussion

In the following we will describe how we obtained the systems that we submitted to the CLEF 2025 evaluation.

Selection procedure As we proposed and tried many different possibilities for the different parts of our SE, we could not test all possible combinations. Therefore we proceeded in the following order:

1. In subsection 5.1 we compared the results of RF and PRF using the best analyzer identified from previous years;
2. In subsection 5.2 we selected the best analyzer, retrieving documents with a fixed base searcher that performed only RF (which gave significant improvements in the previous phase);
3. In subsection 5.3 we compared the rerankers, using the best analyzer found previously and RF;
4. In subsection 5.4 we assessed the need for QE, choosing the best prompts;
5. Finally, in subsection 5.5 we compared the best combination of RF, reranker and QE to decide the systems that we would submit.

We omit for brevity the tests that were necessary to fine tune some of the parameters of both the analyzer and the searcher, like the weights for the different phases and the choice of not using Fuzzy queries nor Proximity ones, but to simply stick to the exact terms. This fine-tuning was done on different months to avoid overfit to a specific document collection. Also, some tests that could have been performed (like testing if using URLs could lead to better results) were not done since they had not yielded good outcomes in previous years.

We note that the order of the tests was dictated by the development of the system, and was not based on any other factor.

Important note on the relevance assessments. During our testing, we identified a series of issues related to the relevance assessments provided by the CLEF. In particular, the initial version of qrels contained duplicates with different relevance values for the same pair of document and query, and the newer version that was released contained some important issues in the first months (from June to September 2022) which led to a big reduction in the performances of our systems. For this reason we only tested our systems in the months from October 2022 to February 2023.

Even in these later months we discovered some problems: inspecting the queries we found out that for some of them the most relevant documents were not really relevant, leading to a nDCG of 0.0 even if the output documents were in some sense related to the query. To give an example, we discovered that query 3 of October 2022, which is about the first world war, had among the most relevant assessed documents one that was only related to cooking. This issue was difficult to spot as `trec_eval`, which is the tool we used to automatically compute the metrics, when computing the mean nDCG for all the queries ignores all the ones that have nDCG equal to 0.0. That's why in the following we conducted statistical analysis only on the queries that had nDCG different from 0.0 for at least one system, ignoring the others.

5.1. Phase 1: Relevance feedback and pseudo relevance feedback

The first test we made regarded the main task of the project: using prior knowledge to leverage the system's performance on new months.

In this phase we tested two different approaches:

- PRF: given a query, we first perform a search with our base system, and then retrieve the words of the top 5 documents of our ranking. We add to the query the 10 most frequent terms of these documents, giving them a weight which is a sort of IDF score (see subsection 3.3.1 for other details), and then re-perform the query;
- RF: this is based on the previous relevance assessments. Given a query, if it has already been performed in the previous months (so if we have some relevant documents), we retrieve 8 of these documents and add the 20 most frequent terms in them to the query, giving them a constant weight of 0.5.

In this phase we tested 4 systems, all with the same analyzer, which was composed of the following components:

- LetterTokenizer;
- ICU Folding filter;
- Lowercase filter;
- Stopwords filter, which used one of the best stoplist of previous year tests plus the 30 most frequent words in the corpus;
- Light stemmer.

The different systems were:

- **Base:** our base system, which doesn't use any form of QE or reranking;

- **Base+PRF**: PRF system, which is our base system but uses PRF;
- **Base+RF**: RF system, which is our base system equipped with the RF mechanism based on previous relevance assessments;
- **Base+RF+PRF**: combined system, which uses both PRF and RF based on previous relevance assessments.

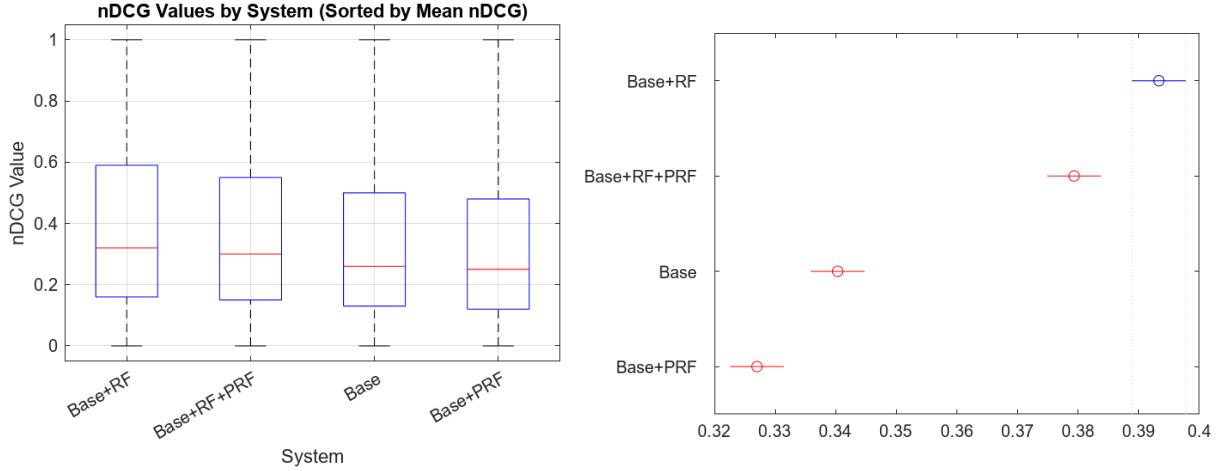


Figure 4: On the left, boxplot of nDCG of the four systems tested in subsection 5.1 on October 2022 dataset, ordered by nDCG. On the right, the result of the Tukey HSD test on the four systems of subsection 5.1.

Results and discussion. We report in Figure 4 the results of this phase only on the month of October 2022, but the same results hold for the other months.

The results of ANOVA (not reported for the sake of space) and Tukey HSD both show that there are some differences among the systems. In particular, Figure 4 shows that using prior knowledge of relevance on the same queries can significantly improve system’s performances: this also doesn’t lead to stale results given that we are not boosting older documents but using their terms to obtain better results. A problem which was really relevant for the base system was the number of queries for which it gave zero results, while with the RF mechanism this number is reduced of up to a factor of 2 in all the months.

The PRF mechanism, on the other hand, gives worse performances than expected, even if used in combination with RF. This is probably due to the fact that it is easier for it to cause a complete topic drift, given that we don’t know if the first elements are in fact relevant.

In the following phases, we will assume that all the tested systems will incorporate the RF mechanism.

5.2. Phase 2: Analyzer

This phase was devoted to the selection of the analyzer: we tried different combinations of the filters, and all of them used the same searcher, which was **Base+RF** of subsection 5.1.

We tested 30 different systems, but the ones which we found most interesting were:

- **Base+RF**: our base system, which used the analyzer described in the previous phase. It corresponds to **Base+RF** of subsection 5.1;
- **Base+Len+RF**: identical to **Base+RF**, but with the addition of a length filter which filtered the tokens with less than 2 characters or more than 15 (the values were obtained after fine tuning);
- **Base+Minim+RF**: identical to **Base+RF**, but with the Minimal stemmer instead of the Light one;
- **Base+Snow+RF**: identical to **Base+RF**, but with the Snowball stemmer instead of the Light one;
- **Base+Lefff+RF**: identical to **Base+RF**, but with our custom Lefff stemmer instead of the Light one;

- **Base+charN+RF**: identical to **Base+RF**, but with the character N -gram filter instead of the Light stemmer, with a value of $N = 5$ (the value was obtained after fine tuning);
- **NoStem+RF**: identical to **Base+RF**, but without any stemmer;
- **Base+wordN+RF**: identical to **Base+RF**, but with the addition of a word N -gram filter which created overlapping sequences of 2 tokens (the value was obtained after fine tuning);
- **Base+Elis+RF**: identical to **Base+RF**, but with the addition of a French elision filter;
- **Base+WOLF+RF**: identical to **Base+RF**, but with the addition of a synonyms filter that added synonyms based on the WOLF database [8];
- **NoStop+RF**: identical to **Base+RF**, but without any stop words filter.

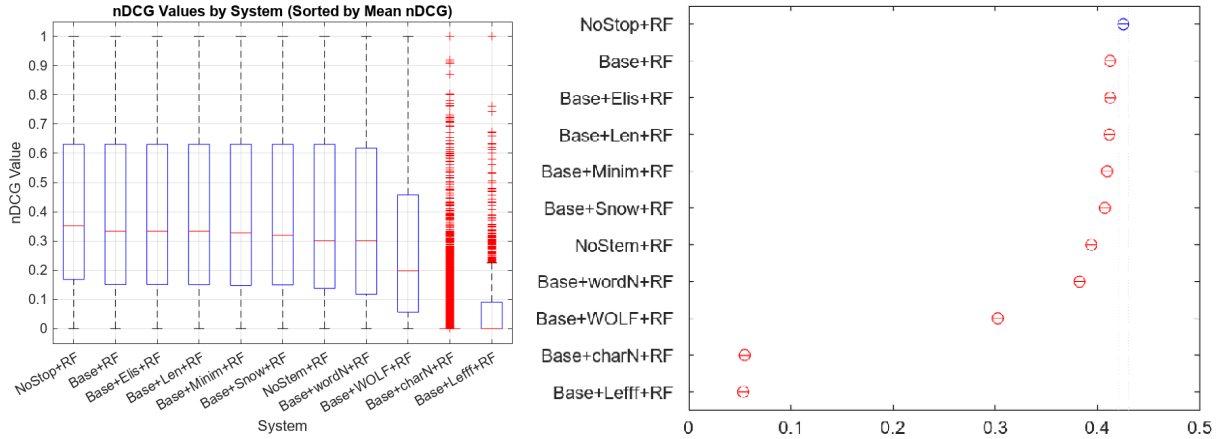


Figure 5: On the left, boxplot of nDCG of the eleven systems of subsection 5.2 on October 2022 dataset, ordered by nDCG. On the right, the result of the Tukey HSD test on the eleven systems of subsection 5.2.

Results and discussion. As in phase 1, we report in Figure 5 the results of this phase only on the month of October 2022, but the same results hold for the other months.

While ANOVA states that the systems are different, Figure 5 shows that the worst systems are the ones with the character N -gram filter (as expected, since in general it performs worse than any other stemmer) and the one with our custom Lefff stemmer, which means that having queries without diacritics worsened a lot its performances.

It is interesting to see that adding synonyms using the WOLF database gives bad results: this is probably due to the poor quality of the database, but also to the fact that adding synonyms can in general lead to topic drift. What’s more, using other filters like French elision and word N -gram seems to deteriorate performances.

We can instead conclude that the best way to proceed is without the use of any stoplist, which gives the best results, probably because our stoplists were not of high quality or because the queries contained a lot of stop words.

Finally, we’d like to note that between phase 1 and phase 2 we performed other tests to achieve better performances for our **Base+RF** system, which gives better results in this phase than in the previous one.

In the following, we will proceed without the use of any stoplist, using **NoStop+RF** as our base system.

5.3. Phase 3: Reranking

In this phase we tested a number of the rerankers that were developed. It is worth noticing that we tested only SBERT, MARCO, MIXEDBREAD, and the one based on previous relevance assessments, dubbed QRELS, while we didn’t have enough resources to test the ones based on ELO and the ones which required the use of an API. We also tried to see if using the full text produced significant improvements.

We tested 8 systems:

- **NoStop+RF**: base system, it doesn't do any reranking, but uses RF. It corresponds to **NoStop+RF** of subsection 5.2;
- **NoStop+QRELS+RF**: identical to **NoStop+RF**, but with a reranker based on previous relevance assessments (since it is not based on document text, there is only one version);
- **NoStop+SBERTFull+RF**: identical to **NoStop+RF**, but with a SBERT reranker based on full text;
- **NoStop+SBERTTerm+RF**: identical to **NoStop+RF**, but with a SBERT reranker based on document body extracted from term vectors;
- **NoStop+MARCOFull+RF**: identical to **NoStop+RF**, but with a MARCO reranker based on full text;
- **NoStop+MARCOTerm+RF**: identical to **NoStop+RF**, but with a MARCO reranker based on document body extracted from term vectors;
- **NoStop+MBREADTerm+RF**: identical to **NoStop+RF**, but with a MIXEDBREAD reranker based on full text (we weren't able to test this model with body extracted from term vectors due to the heavy latency, but as the ones with document extracted from term vectors perform in general worse, probably it wouldn't have succeeded).

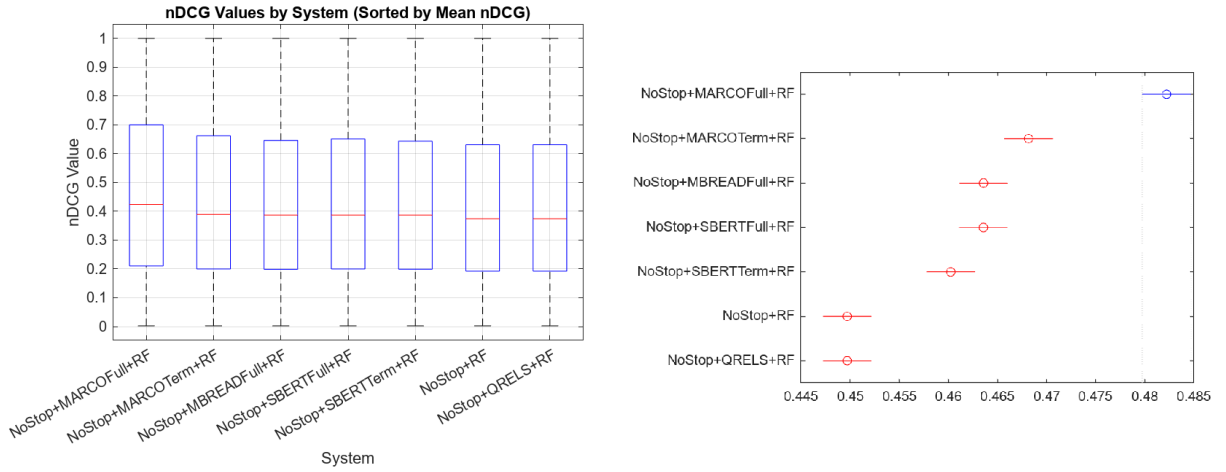


Figure 6: On the left, boxplot of nDCG of the 7 systems of subsection 5.3 on October 2022 dataset, ordered by nDCG. On the right, the result of the Tukey HSD test on the 8 systems of subsection 5.3.

Results and discussion. Figure 6, which as usual displays only the results for October 2022, shows that there are some statistical differences between systems. In general, systems that use a reranker based on full text perform better, as the terms of the text are not stemmed and therefore easier to use (by LLMs but also by other DL algorithms). Also, using the right reranker increases drastically the nDCG of the overall system.

The most disappointing reranker is the one based on previous relevance assessments, which performs worse than the base system: this could be caused by different factors, such as the poor quality of the relevance assessments or the fact that boosting older documents may lead to stale results.

On the other hand, the best reranker is MARCO, which is significantly better than the others, other than faster. That's why in the following we will focus on it.

5.4. Phase 4: Query expansion

The fourth phase was about deciding which prompt is better for QE: as expressed in subsubsection 3.3.1 we have 4 prompts to test, and therefore 4 systems.

Due to the high latency of the QE mechanism, we expanded only the first 200 queries of December 2022 with all the prompts to see which one could work better. In this phase we didn't use any reranker.

The 5 systems are:

- **NoStop+RF**: base system, identical to **NoStop+RF** of subsection 5.2;
- **NoStop+RF+QE1**: base system with prompt 1;
- **NoStop+RF+QE2**: base system with prompt 2;
- **NoStop+RF+QE3**: base system with prompt 3;
- **NoStop+RF+QE4**: base system with prompt 4.

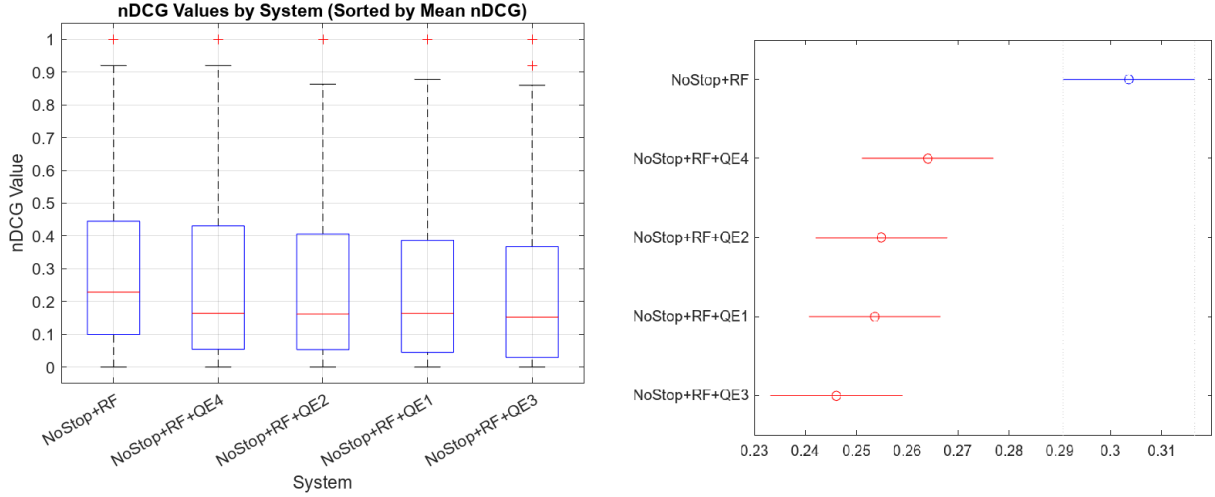


Figure 7: On the left, boxplot of nDCG of the 5 systems on the first 200 queries of December 2022, ordered by nDCG. On the right, the result of the Tukey HSD test on the 5 systems.

Results and discussion. Figure 7 shows that, even if the best prompt is prompt 4, there are no statistically relevant differences with the other ones, as they all perform worse than the base system. This could be due to the fact that the prompts required further refinement or that we could use better LLM models. In the final phase we will test both prompt 1 and 4, which represent two of the most promising ways of QE: asking for related keywords and asking to express explicitly the information need of the user.

5.5. Phase 5: Final comparison

This comparison was set after all the previous steps, using the best possibilities for each one. In this phase we always used the full text reranking. We tested:

- **NoStop+RF**: base system with RF;
- **NoStop+QE4**: base system with QE adopting prompt 4;
- **NoStop+QE1**: base system with QE adopting prompt 1;
- **NoStop+RF+QE4**: base system with RF and QE adopting prompt 4;
- **NoStop+RF+QE1**: base system with RF and QE adopting prompt 1;
- **NoStop+MARCO+RF+QE4**: base system with RF, MARCO reranker and QE adopting prompt 4;
- **NoStop+MARCO+RF+QE1**: base system with RF, MARCO reranker and QE adopting prompt 1;
- **NoStop+MARCO**: base system with MARCO reranker;
- **NoStop+MARCO+RF**: base system with MARCO reranker and RF;
- **NoStop+MBREAD+RF**: base system with MIXEDBREAD reranker and RF;
- **NoStop+SBERT+RF**: base system with SBERT reranker and RF;
- **NoStop+QRELS+RF**: base system with reranker based on previous relevance assessments and RF;

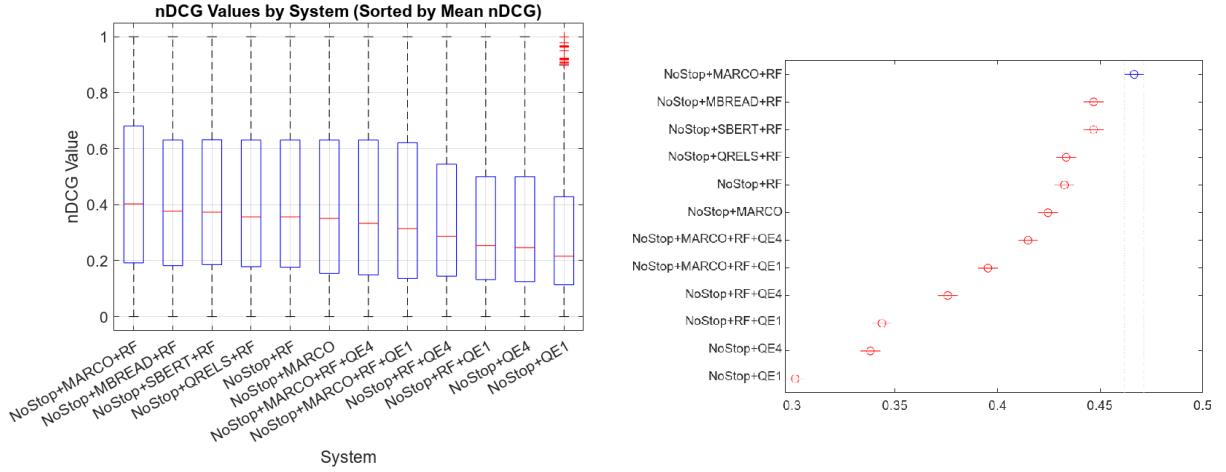


Figure 8: On the left, boxplot of nDCG of the 12 systems on October 2022 dataset, ordered by nDCG. On the right, the result of the Tukey HSD test on the 12 systems.

Results and discussion. While ANOVA states that the systems are statistically different, Figure 8 clearly shows the advantage of using a reranking phase, given that the top 4 systems all use relevance feedback and a reranker. We can also see that after fine tuning the weights given to the reranker score, **NoStop+QRELS+RF** finally beats **NoStop+RF**.

Other relevant systems are the ones that use the RF mechanism or MARCO alone, which beat all the form of QE. This is probably due to the fact that the expanded terms are highly likely to cause a query drift. We can also see that the fourth prompt, which asks to give the information need of the user, is statistically better than the first one, which asks for related terms.

Overall, our best system achieves an nDCG of 0.3853 on the month of October 2022, while improving to 0.4259 in November 2022, 0.4332 in December 2022 and even 0.4425 on January 2023, while decreasing to 0.4125 in February 2023. This improvement is probably highly correlated to the quality of the relevance assessments, and it forces us to not consider the RnD as a true estimate of the robustness of our system to temporal changes.

6. Conclusions and Future Work

6.1. Our achievements

In this work, we have demonstrated that RF based on relevance assessments from previous months can achieve performance levels comparable to those of dedicated reranking components. This validates RF as a lightweight yet effective enhancement strategy. Furthermore, the combination of RF with a DL-based reranker yielded the highest nDCG improvements, confirming the strength of these two strategies. We also empirically confirmed that PRF may introduce topic drift and degrade performance.

Conversely, our investigation into LLM-based QE was less successful: the inclusion of expanded queries led to performance degradation. Nevertheless, we believe this line of research remains promising and that better results could be achieved through improved prompt engineering.

6.2. Further development

Many of the ideas we proposed and tried allow further fine tuning, experimentation and development.

LEFFF-Lemmatizer The current lemmatizer is probably too strict and does not allow for enough flexibility in the matching process, in particular when dealing with accents. In the future, this limitation may be overcome by preprocessing the LEFFF lexicon removing diacritics prior to lookup, or by incorporating fuzzy matching techniques.

Reranking The RF-based reranking technique we tried was mostly a proof of concept: it demonstrates that there is some information to be gained from old versions. Something that could work well is to use more elaborate freshness weighting.

In the reranking phase, as highlighted previously, we were limited by the computational resources available to us. Even having better devices, fine-tuning of the many parameters, and prompts could be beneficial to the final performance.

Query expansion This last observation is also true for the QE phase, which heavily used LLM: the use of LLMs with advanced language understanding and multilingual capabilities can improve results. Further research should also investigate prompt engineering strategies for QE, such as few-shot prompting. This technique involves providing language models with a small number of input–output examples to guide their behavior.

7. Group Members Contribution

Giovanni Gaio Proposed the LLM-based ELO technique for reranking, and more in general researched and implemented the different DL based techniques for reranking. He also looked after of many stylistic and technical details in the $\LaTeX$ final report, after writing parts of the introduction, methodology and conclusions.

Filippo Mazzarotto Focused on the Analyzer component, researching and designing its general processing pipeline, validating the various steps with unit testing. Researched and developed the project’s custom LEFFL Lemmatizer. Contributed to group organization and task prioritization (through Kanban board) and curated figures for the final report.

Matteo Meneghin Primarily focused on the query expansion component, in particular the python script for testing different prompts. Supported group organization, contributed to the final report, particularly the query expansion subsection.

Enrico Saro Primarily focused on the query expansion component, including the design and testing of prompts for the LLM. Supported group organization, reviewed previous years’ work and implemented standard filters within the Analyzer component. Contributed to the final report, particularly the abstract, introduction, conclusions and the query expansion subsection.

Francesco Visonà After reviewing the previous years’ work, developed the overall SE pipeline, including the different parts of the Analyzer components (in particular the synonyms and OpenNLP filters), introducing parallelism and utility components needed to parse and correct the results provided by the organizers, test the code and make the system more flexible. Focused on the relevance feedback and pseudo relevance feedback phases, while also developing some API based rerankers. Chose and conducted all the experiments, including the query expansion phase, and produced the statistical analysis of the results. Contributed to the final report by including results and discussion section, the overall pipeline and the relevance feedback subsection.

References

- [1] CLEF Organizers, Longeval lab - clef 2025, 2025. URL: <https://clef-longeval.github.io/>, accessed: 2025-05-02.
- [2] A. Sharma, Practical Apache Lucene 8. Uncover the Search Capabilities of Your Application, Apress Media, New York, USA, 2020.
- [3] J. Keller, T. Breuer, P. Schaer, Leveraging prior relevance signals in web search, in: G. Faggioli, N. Ferro, P. Galuscáková, A. G. S. de Herrera (Eds.), Working Notes of the Conference and Labs of the Evaluation Forum (CLEF 2024), Grenoble, France, 9-12 September, 2024, volume 3740 of *CEUR*

Workshop Proceedings, CEUR-WS.org, 2024, pp. 2396–2406. URL: <https://ceur-ws.org/Vol-3740/paper-220.pdf>.

- [4] L. Cazzador, F. Faveri, F. Franceschini, L. Pamio, S. Piron, N. Ferro, Seupd@ clef: team mouse on enhancing search engines effectiveness with large language models, Faggioli et al.[12] (2024).
- [5] R. Jagerman, H. Zhuang, Z. Qin, X. Wang, M. Bendersky, Query expansion by prompting large language models, 2023. URL: <https://arxiv.org/abs/2305.03653>. arXiv: 2305.03653.
- [6] Y. Lei, Y. Cao, T. Zhou, T. Shen, A. Yates, Corpus-steered query expansion with large language models, in: Y. Graham, M. Purver (Eds.), *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*, Association for Computational Linguistics, St. Julian's, Malta, 2024, pp. 393–401. URL: <https://aclanthology.org/2024.eacl-short.34/>.
- [7] Unicode Consortium, Unicode Standard Annex #29: Unicode Text Segmentation, Technical Report UAX #29, Revision 45, Unicode Consortium, 2024. URL: <https://unicode.org/reports/tr29/>, latest version available at <https://unicode.org/reports/tr29/>. This version corresponds to Unicode 16.0.0. Editor: Josh Hadley.
- [8] B. Sagot, D. Fišer, Building a free French wordnet from multilingual resources, in: *Proceedings of the OntoLex 2008 Workshop*, Marrakech, Morocco, 2008.
- [9] B. Sagot, The lefff, a freely available and large-coverage morphological and syntactic lexicon for french, 7th international conference on Language Resources and Evaluation (LREC 2010) (2010).
- [10] F. Galli, M. Rigobello, M. Schibuola, R. Zuech, N. Ferro, et al., Seupd@ clef: team iris on temporal evolution of query expansion and rank fusion techniques applied to cross-encoder re-rankers, Faggioli et al.[12] (2024).
- [11] S. E. Robertson, U. Zaragoza, The Probabilistic Relevance Framework: BM25 and Beyond, *Foundations and Trends in Information Retrieval (FnTIR)* 3 (2009) 333–389.
- [12] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, et al., The llama 3 herd of models, 2024. URL: <https://arxiv.org/abs/2407.21783>. arXiv: 2407.21783.
- [13] Meta, The llama 4 herd: The beginning of a new era of natively multimodal ai innovation, 2025. URL: <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, accessed: 2025-05-04.
- [14] N. Reimers, I. Gurevych, Sentence-bert: Sentence embeddings using siamese bert-networks, in: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2019. URL: <http://arxiv.org/abs/1908.10084>.
- [15] N. Reimers, I. Gurevych, Making monolingual sentence embeddings multilingual using knowledge distillation, in: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2020. URL: <https://arxiv.org/abs/2004.09813>.
- [16] A. Shakir, D. Koenig, J. Lipp, S. Lee, Boost your search with the crispy mixedbread rerank models, 2024. URL: <https://www.mixedbread.ai/blog/mxbai-rerank-v1>.
- [17] P. G. R. Deveaud, G. Gonzalez-Saez, P. Mulhem, L. Goeuriot, F. Piroi, M. Popel, Longeval-retrieval: French-english dynamic test collection for continuous web search evaluation, 2023. URL: <https://arxiv.org/abs/2303.03229>. arXiv: 2303.03229.
- [18] T. Fink, F. Piroi, R. Devaud, P. Galuščáková, G. Gonzalez-Saez, D. Iommi, P. Mulhem, L. Goeuriot, M. Popel, A. El-Ebshihy, Longeval 2025 web retrieval train collection, 2025. doi:10.48436/th5h0-g5f51.
- [19] C. D. Manning, P. Raghavan, H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, Cambridge, UK, 2008.